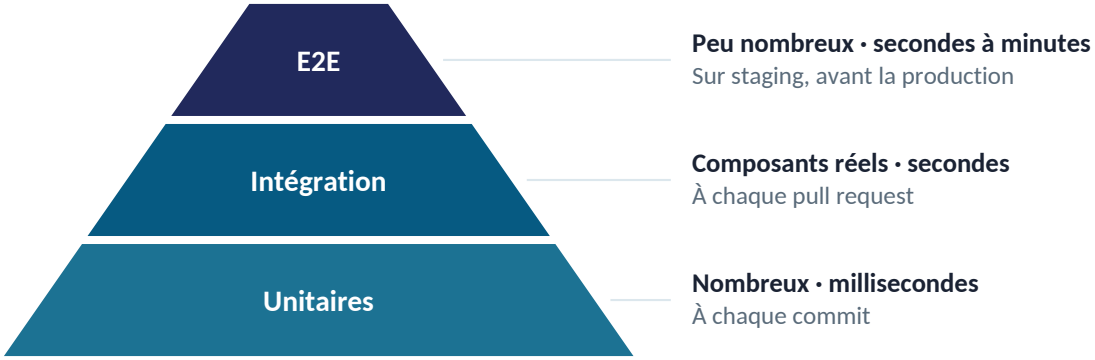
Atelier 1 : la pyramide de tests sur une API

Une seule application fil rouge, trois niveaux de tests : on construit la pyramide de bas en haut.

Objectif

- Écrire les **3 niveaux de tests** : unitaires, d'intégration et fonctionnels (E2E).
- Mesurer la **couverture de code** et atteindre au moins **80 %**.
- Comprendre **quand lancer chaque niveau** dans le pipeline CI.

La pyramide des tests



Plus on monte, plus les tests sont lents, fragiles et coûteux : on en écrit donc moins. L'application fil rouge est une mini-boutique Node.js : un panier, une API utilisateurs branchée sur PostgreSQL et une page de connexion.

Déroulé

Partie	Contenu	Outils
1	Tests unitaires, mock et couverture de code	Jest
2	Tests d'intégration avec une vraie base de données, puis CI	Supertest, PostgreSQL, GitHub Actions
3	Test fonctionnel de bout en bout dans un navigateur	Playwright

Comment utiliser ce document

Chaque étape suit le même déroulement, du début à la fin de l'atelier.

Bloc	Rôle
Consigne	Ce que vous devez faire et pourquoi.
Bloc de code sombre	Ce qu'il faut taper ou écrire. Le libellé indique le fichier ou « terminal ».
Résultat attendu	Ce que vous devez voir à l'écran. Les valeurs (durées, numéros de version) peuvent différer légèrement.
Point de contrôle	Cases à cocher. Ne passez à l'étape suivante que lorsque toutes sont validées.
En cas de problème	Les erreurs les plus fréquentes de la partie et leur solution.

Règles de travail

- Tapez le code vous-même plutôt que de le copier aveuglément : vous comprendrez mieux ce que fait chaque ligne.
- **Un commit par étape ou par groupe d'étapes**, avec un message lisible. L'historique est évalué.
- Lancez les commandes à la **racine du projet** (le dossier qui contient `package.json`).
- En cas d'erreur, lisez le message en entier : la première ligne indique presque toujours le fichier et la ligne en cause.

Prérequis

- **Node.js 20 ou plus** et npm.
- **Docker** avec Docker Compose (le plugin `docker compose`).
- **Git** et un compte GitHub (pour la partie CI).
- Un éditeur de code (VS Code conseillé) et un navigateur récent.
- Les bases de Jest vues en cours (`test`, `expect`).

Mise en place

À faire une seule fois, avant la partie 1.

Étape 1 · Vérifier l'environnement

Ouvrir un terminal et vérifier que chaque outil est installé et assez récent.

```
terminal
node -v
npm -v
docker --version
docker compose version
git --version
```

```
résultat attendu (indicatif)
v20.11.1
10.2.4
Docker version 26.1.0, build 9714adc
Docker Compose version v2.27.0
git version 2.43.0
```

Point de contrôle

- ☐ Le numéro de version de Node commence par **v20** ou plus.
- ☐ Les cinq commandes affichent une version (aucune ne répond « command not found »).

Étape 2 · Créer le projet

On crée le dossier, on l'initialise comme dépôt Git et comme projet npm. L'option `-y` accepte toutes les valeurs par défaut.

```
terminal
mkdir atelier-tests
cd atelier-tests
git init
npm init -y
```

Vous devez voir un fichier `package.json` dans le dossier (commande `ls`).

Étape 3 · Installer les dépendances

```
terminal
npm i express pg
npm i -D jest supertest @playwright/test
```

L'option `-D` range un paquet dans les dépendances de développement (outils de test, absents de la production).

Paquet	Rôle dans l'atelier
`express`	Serveur web et API.
`pg`	Client PostgreSQL pour Node.js.
`jest`	Lanceur de tests et bibliothèque d'assertions (unitaires et intégration).
`supertest`	Envoie des requêtes HTTP à l'API sans démarrer de vrai serveur.
`@playwright/test`	Pilote un vrai navigateur pour les tests de bout en bout.

Étape 4 · Créer l'arborescence

```
terminal
mkdir -p src tests/unit tests/integration e2e public .github/workflows
```

À la fin de l'atelier, votre projet ressemblera à ceci. Vous créerez chaque fichier au fil des étapes.

```
arborescence cible
atelier-tests/
├── .github/workflows/ci.yml
├── e2e/login.spec.js
├── public/login.html
├── public/dashboard.html
├── src/panier.js
├── src/mailer.js
├── src/commande.js
├── src/db.js
├── src/app.js
├── src/server.js
├── tests/unit/panier.test.js
├── tests/unit/commande.test.js
├── tests/integration/users.test.js
├── docker-compose.yml
├── playwright.config.js
├── .gitignore
└── package.json
```

Étape 5 · Configurer package.json

Ouvrir `package.json`. Remplacer la section `scripts` (elle contient un script `test` par défaut) et ajouter la section `jest` juste en dessous. Ne touchez pas aux autres champs. Attention à la syntaxe JSON : pas de commentaire et une virgule entre deux sections.

```
package.json (sections à écrire ; vos autres champs restent en place)
{
  "name": "atelier-tests",
  "version": "1.0.0",
  "scripts": {
    "start": "node src/server.js",
    "test": "jest",
    "test:unit": "jest tests/unit --coverage",
    "test:int": "jest tests/integration --runInBand",
    "test:e2e": "playwright test"
  },
  "jest": {
    "testPathIgnorePatterns": ["/node_modules/", "/e2e/"]
  }
}
```

Script	Ce qu'il fait
`npm start`	Démarre l'application sur le port 3000.
`npm run test:unit`	Lance uniquement les tests unitaires et calcule la couverture (`--coverage`).
`npm run test:int`	Lance les tests d'intégration, un fichier à la fois (`--runInBand`).
`npm run test:e2e`	Lance les scénarios Playwright.

Pourquoi la section jest ?

- Elle empêche Jest de ramasser les fichiers du dossier `e2e/` : Jest et Playwright reconnaissent tous deux les fichiers `*.spec.js`, mais ne s'exécutent pas de la même façon.

Étape 6 · Fichier .gitignore et premier commit

Ne jamais versionner les dossiers générés. Créer `.gitignore` à la racine :

```
.gitignore
node_modules/
coverage/
playwright-report/
test-results/
```

```
terminal
git add .
git commit -m "chore: initialise le projet"
git log --oneline
```

Point de contrôle

- ☐ `package.json` contient les 5 scripts et la section `jest`.
- ☐ L'arborescence contient les dossiers `src`, `tests`, `e2e`, `public`.
- ☐ `git log --oneline` affiche au moins un commit.
- ☐ `node_modules/` n'apparaît pas dans `git status`.

1 Tests unitaires

Outil : Jest

Un test unitaire vérifie **une unité de code isolée** : une fonction, sans base de données ni réseau. Il est rapide (quelques millisecondes), déterministe et localise immédiatement l'erreur.

À retenir

- Pattern **AAA** : **Arrange** (préparer les données), **Act** (exécuter la fonction), **Assert** (vérifier le résultat).
- Un **mock** est un faux objet qui remplace une dépendance (envoi d'e-mail, base, API externe) pour isoler le code testé.
- Un test doit toujours être vu **échouer** au moins une fois : sinon on ignore s'il teste vraiment quelque chose.

Étape 1 · Écrire la fonction à tester

Créer `src/panier.js`. La fonction `total` additionne prix × quantité pour chaque ligne du panier.

```
src/panier.js
const total = (items) => items.reduce((s, i) => s + i.prix * i.qte, 0);

module.exports = { total };
```

Étape 2 · Écrire un premier test

Créer `tests/unit/panier.test.js`. Chaque test est découpé en trois temps (AAA).

```
tests/unit/panier.test.js
const { total } = require('../../src/panier');

describe('total()', () => {
  test('calcule le total du panier', () => {
    // Arrange : préparer les données
    const items = [{ prix: 10, qte: 2 }, { prix: 5, qte: 1 }];
    // Act : exécuter la fonction
    const resultat = total(items);
    // Assert : vérifier le résultat
    expect(resultat).toBe(25);
  });
});
```

```
terminal
npm run test:unit
```

```
résultat attendu (indicatif)
PASS tests/unit/panier.test.js
  total()
    ✓ calcule le total du panier (2 ms)

Test Suites: 1 passed, 1 total
Tests:       1 passed, 1 total
```

Le tableau de couverture s'affiche aussi sous ce résultat : on l'étudiera à l'étape 6.

Étape 3 · Tester les cas limites

Un bon jeu de tests ne vérifie pas seulement le cas normal. Ajouter, **à l'intérieur du** `describe`, après le premier test :

```
tests/unit/panier.test.js (à ajouter dans le describe)
  test('retourne 0 pour un panier vide', () => {
    expect(total([])).toBe(0);
  });

  test('ignore une ligne dont la quantité est 0', () => {
    const items = [{ prix: 10, qte: 0 }, { prix: 5, qte: 2 }];
    expect(total(items)).toBe(10);
  });
```

```
terminal
npm run test:unit
```

Les trois tests doivent passer. Faites votre premier commit de la partie :

Commit conseillé : `git add . && git commit -m "test: ajoute les tests unitaires du panier"`

Étape 4 · Exiger une erreur sur un prix négatif (rouge, puis vert)

On applique la démarche « rouge → vert » : on écrit d'abord le test, on le voit échouer, puis on corrige le code. Ajouter ce test dans le `describe` :

```
tests/unit/panier.test.js (à ajouter dans le describe)
  test('refuse un prix négatif', () => {
    const items = [{ prix: -5, qte: 1 }];
    expect(() => total(items)).toThrow('Prix négatif');
  });
```

```
terminal
npm run test:unit
```

```
résultat attendu (indicatif)
FAIL tests/unit/panier.test.js
  ● total() › refuse un prix négatif

    expect(received).toThrow(expected)

    Expected substring: "Prix négatif"
    Received function did not throw
```

Le test est **rouge** : c'est normal, la fonction accepte encore les prix négatifs. Corriger maintenant `src/panier.js` :

```
src/panier.js (version corrigée)
const total = (items) =>
  items.reduce((s, i) => {
    if (i.prix < 0) throw new Error('Prix négatif');
    return s + i.prix * i.qte;
  }, 0);

module.exports = { total };
```

```
terminal
npm run test:unit
```

Les **4 tests** doivent maintenant être **verts**.

Commit conseillé : `git add . && git commit -m "feat: refuse un prix négatif dans le panier"`

Étape 5 · Isoler une dépendance avec un mock

Valider une commande envoie un e-mail de confirmation. Dans un test unitaire, on ne veut **jamais** envoyer un vrai e-mail : on remplace le mailer par un mock. Créer d'abord les deux fichiers de l'application :

```
src/mailer.js (simule un service d'e-mail non configuré)
exports.envoyer = async () => {
  throw new Error('SMTP non configuré');
};
```

```
src/commande.js
const { total } = require('./panier');
const mailer = require('./mailer');

async function validerCommande(items, email) {
  if (items.length === 0) throw new Error('Panier vide');
  const montant = total(items);
  await mailer.envoyer(email, `Commande validée : ${montant} €`);
  return { montant };
}

module.exports = { validerCommande };
```

Sans mock, un test de `validerCommande` échouerait avec l'erreur « SMTP non configuré ». Créer `tests/unit/commande.test.js` :

```
tests/unit/commande.test.js
jest.mock('../../src/mailer');

const mailer = require('../../src/mailer');
const { validerCommande } = require('../../src/commande');

beforeEach(() => {
  jest.clearAllMocks();
});

test('envoie un e-mail de confirmation et retourne le montant', async () => {
  mailer.envoyer.mockResolvedValue();
  const items = [{ prix: 10, qte: 2 }, { prix: 5, qte: 1 }];

  const resultat = await validerCommande(items, 'client@test.fr');

  expect(resultat.montant).toBe(25);
  expect(mailer.envoyer).toHaveBeenCalledTimes(1);
  expect(mailer.envoyer).toHaveBeenCalledWith('client@test.fr', expect.stringContaining('25'));
});

test("refuse un panier vide et n'envoie aucun e-mail", async () => {
  await expect(validerCommande([], 'client@test.fr')).rejects.toThrow('Panier vide');
  expect(mailer.envoyer).not.toHaveBeenCalled();
});
```

Élément	Rôle
`jest.mock('../../src/mailer')`	Remplace toutes les fonctions du module par des fonctions simulées qui ne font rien.
`mockResolvedValue()`	Indique que la fonction simulée réussit (promesse résolue).
`toHaveBeenCalledTimes(1)`	Vérifie que l'e-mail a été envoyé exactement une fois.

Élément	Rôle
`toHaveBeenCalledWith(...)`	Vérifie les arguments : la bonne adresse et un message contenant le montant.
`jest.clearAllMocks()`	Remet les compteurs d'appels à zéro entre deux tests.

```
terminal
npm run test:unit
```

Vous devez voir **deux fichiers de test** et **6 tests** passés au total.

Commit conseillé : `git add . && git commit -m "test: ajoute les tests unitaires de la commande avec mock"`

Étape 6 · Mesurer la couverture

La couverture indique quelle part du code est exécutée par les tests. L'objectif est **au moins 80 %** de lignes. Le tableau s'affiche à chaque `npm run test:unit` :

```
résultat attendu (indicatif)
-------------|---------|----------|---------|---------|
File         | % Stmts | % Branch | % Funcs | % Lines |
-------------|---------|----------|---------|---------|
All files    |     100 |      100 |     100 |     100 |
 commande.js |     100 |      100 |     100 |     100 |
 panier.js   |     100 |      100 |     100 |     100 |
-------------|---------|----------|---------|---------|
```

Le fichier `mailer.js` peut apparaître avec une couverture partielle ou être absent : c'est normal, il est remplacé par un mock. Les fichiers non chargés par un test unitaire (`app.js`, `db.js`) ne sont pas comptés.

Lire le rapport détaillé : ouvrir `coverage/lcov-report/index.html` dans le navigateur. Les lignes en rouge ne sont jamais exécutées par un test.

Expérience à faire

1. Mettre en commentaire le test « refuse un panier vide » dans `commande.test.js`.
2. Relancer `npm run test:unit` : la couverture de branches de `commande.js` baisse.
3. Rouvrir le rapport HTML et repérer la ligne devenue rouge, puis remettre le test.

Point de contrôle

- ☐ 6 tests passent, dans 2 fichiers de test.
- ☐ La couverture globale est **≥ 80 %** (capture d'écran à conserver pour le livrable).
- ☐ Vous avez vu le test du prix négatif échouer avant de le corriger.
- ☐ Le rapport HTML de couverture s'ouvre dans le navigateur.

Questions

1. Pourquoi mocker le mailer plutôt que de laisser le vrai envoi ?
2. Un test unitaire peut-il être « non déterministe » ? Donner un exemple (date, valeur aléatoire).

En cas de problème

Symptôme	Cause probable	Solution
`Cannot find module '../../src/panier'`	Mauvais chemin relatif ou nom de fichier différent.	Vérifier l'orthographe et la profondeur : depuis `tests/unit/`, il faut remonter de 2 niveaux.
`No tests found`	Le fichier de test ne se termine pas par `.test.js` ou n'est pas dans `tests/unit`.	Renommer le fichier.
`SMTP non configuré` dans un test	L'appel `jest.mock` manque ou ne vise pas le bon fichier.	Mettre `jest.mock('../../src/mailer')` tout en haut du fichier.
Couverture sous 80 %	Des branches du code ne sont pas testées.	Ouvrir le rapport HTML et écrire le test des lignes rouges.

Commit conseillé : `git add . && git commit -m "test: atteint 80 % de couverture"`

2 Tests d'intégration

Outils : Supertest, PostgreSQL, GitHub Actions

Ici, **aucun mock** : l'API, la couche d'accès aux données et une vraie base PostgreSQL fonctionnent ensemble. On détecte ainsi les erreurs d'assemblage que les tests unitaires ne voient pas (requête SQL fausse, mauvais code HTTP).

> **À retenir**
>
> - La base de test tourne dans un **conteneur Docker** jetable, jamais sur une base de production.
> - Chaque test repart de **données connues** : on recrée la table avant chaque test.
> - Supertest appelle l'application **directement**, sans ouvrir de port : c'est pourquoi `app` est exporté sans `listen`.

Étape 1 · Démarrer la base de test

Créer `docker-compose.yml` à la racine. Il décrit un seul service : une base PostgreSQL avec un utilisateur, un mot de passe et une base de test.

```
docker-compose.yml
services:
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: test
      POSTGRES_PASSWORD: test
      POSTGRES_DB: testdb
    ports:
      - "5432:5432"
```

```
terminal
docker compose up -d
docker compose ps
```

```
résultat attendu (indicatif)
NAME                 SERVICE   STATUS         PORTS
atelier-tests-db-1   db        Up 5 seconds   0.0.0.0:5432->5432/tcp
```

Vérifier que la base répond vraiment :

```
terminal
docker compose exec db psql -U test -d testdb -c "SELECT 1;"
```

```
résultat attendu (indicatif)
 ?column?
----------
        1
(1 row)
```

Étape 2 · Connecter l'application à la base

Créer `src/db.js`. Un **Pool** gère un ensemble de connexions réutilisables. L'adresse vient de la variable d'environnement `DATABASE_URL`, avec une valeur par défaut pour le poste local.

```
src/db.js
const { Pool } = require('pg');

module.exports = new Pool({
  connectionString:
    process.env.DATABASE_URL || 'postgres://test:test@localhost:5432/testdb',
});
```

Étape 3 · Écrire l'API

Créer `src/app.js`. Lire attentivement la route : elle valide l'entrée, interroge la base avec une **requête paramétrée** (`$1`, jamais de concaténation de texte) et renvoie le bon code HTTP.

```
src/app.js
const express = require('express');
const db = require('./db');

const app = express();
app.use(express.json());
app.use(express.static('public'));

app.get('/users/:id', async (req, res) => {
  const id = Number(req.params.id);
  if (!Number.isInteger(id)) {
    return res.status(400).json({ erreur: 'id invalide' });
  }
  const { rows } = await db.query('SELECT id, nom FROM users WHERE id = $1', [id]);
  if (!rows[0]) {
    return res.status(404).json({ erreur: 'introuvable' });
  }
  res.json(rows[0]);
});

module.exports = app;
```

```
src/server.js
require('./app').listen(3000, () => {
  console.log('Serveur prêt sur http://localhost:3000');
});
```

Essai rapide à la main (la base n'a pas encore de table, mais l'identifiant invalide est rejeté avant d'y toucher) :

```
terminal
npm start
# dans un second terminal :
curl -i http://localhost:3000/users/abc
```

```
résultat attendu (indicatif)
HTTP/1.1 400 Bad Request
Content-Type: application/json; charset=utf-8

{"erreur":"id invalide"}
```

Arrêter le serveur avec `Ctrl + C` dans le premier terminal.

Étape 4 · Écrire le premier test d'intégration

Créer `tests/integration/users.test.js`. Le point important est le `beforeEach` : avant **chaque** test, on supprime et recrée la table avec une seule ligne connue.

```
tests/integration/users.test.js
const request = require('supertest');
const app = require('../../src/app');
const db = require('../../src/db');

beforeEach(async () => {
  await db.query('DROP TABLE IF EXISTS users');
  await db.query('CREATE TABLE users (id SERIAL PRIMARY KEY, nom TEXT NOT NULL)');
  await db.query("INSERT INTO users (nom) VALUES ('Alice')");
});

afterAll(async () => {
  await db.end();
});

describe('GET /users/:id', () => {
  test("retourne 200 et l'utilisateur existant", async () => {
    const res = await request(app).get('/users/1');

    expect(res.status).toBe(200);
    expect(res.body.nom).toBe('Alice');
  });
});
```

Élément	Rôle
`beforeEach`	S'exécute avant chaque test : garantit un état de départ identique.
`afterAll → db.end()`	Ferme les connexions à la fin. Sans cela, Jest reste bloqué et n'affiche jamais « terminé ».
`request(app).get(...)`	Envoie une requête GET à l'application, comme le ferait un navigateur.
`res.status / res.body`	Code HTTP et corps JSON de la réponse.

```
terminal
npm run test:int
```

```
résultat attendu (indicatif)
PASS tests/integration/users.test.js
  GET /users/:id
    ✓ retourne 200 et l'utilisateur existant (45 ms)

Test Suites: 1 passed, 1 total
Tests:       1 passed, 1 total
```

Étape 5 · Tester les erreurs et l'injection SQL

Ajouter ces trois tests **dans le** `describe`, après le premier. Le dernier simule une attaque par injection SQL : il prouve que la validation et la requête paramétrée protègent la table.

```
tests/integration/users.test.js (à ajouter dans le describe)
  test('retourne 404 pour un utilisateur inconnu', async () => {
    const res = await request(app).get('/users/999');
    expect(res.status).toBe(404);
  });

  test('retourne 400 pour un identifiant invalide', async () => {
    const res = await request(app).get('/users/abc');
    expect(res.status).toBe(400);
  });

  test("résiste à une tentative d'injection SQL", async () => {
    const attaque = encodeURIComponent('1; DROP TABLE users;--');
    const res = await request(app).get('/users/' + attaque);

    expect(res.status).toBe(400);
    // la table doit toujours exister et contenir sa ligne
    const { rows } = await db.query('SELECT count(*) FROM users');
    expect(Number(rows[0].count)).toBe(1);
  });
```

```
terminal
npm run test:int
```

Vous devez obtenir **4 tests verts**. Faites un commit :

Commit conseillé : `git add . && git commit -m "test: ajoute les tests d'integration de l'API users"`

> **Expérience à faire**
>
> 1. Mettre en commentaire la ligne `DROP TABLE IF EXISTS users` dans le `beforeEach`, puis relancer deux fois.
> 2. Observer l'erreur (la table existe déjà) : c'est la réponse à la question 2 ci-dessous.
> 3. Remettre la ligne.

Étape 6 · Automatiser dans la CI (GitHub Actions)

On veut que les tests tournent automatiquement à chaque modification. Créer `.github/workflows/ci.yml`. Le job `unit` tourne à chaque commit ; le job `integration` seulement sur les pull requests, avec une base PostgreSQL démarrée comme **service** du job.

```
.github/workflows/ci.yml
name: CI
on: [push, pull_request]

jobs:
  unit:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: npm
      - run: npm ci
      - run: npm run test:unit

  integration:
    if: github.event_name == 'pull_request'
    runs-on: ubuntu-latest
    services:
      db:
        image: postgres:16-alpine
        env:
          POSTGRES_USER: test
          POSTGRES_PASSWORD: test
          POSTGRES_DB: testdb
        ports:
          - 5432:5432
        options: >-
          --health-cmd "pg_isready -U test"
          --health-interval 5s
          --health-timeout 5s
          --health-retries 10
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: npm
      - run: npm ci
      - run: npm run test:int
```

Élément	Rôle
`on: [push, pull_request]`	Déclenche le workflow à chaque push et à chaque pull request.
`if: github.event_name == 'pull_request'`	Le job d'intégration (plus lent) ne tourne que sur les pull requests.
`services: db`	Démarre un conteneur PostgreSQL à côté du job, joignable sur localhost:5432.
`--health-cmd pg_isready`	Attend que la base soit prête avant de lancer les tests.
`npm ci`	Installe exactement les versions du `package-lock.json` : plus reproductible que `npm i`.

Pousser le projet sur GitHub : créer un dépôt vide nommé `atelier-tests` sur github.com, puis :

```
terminal
git add .
git commit -m "ci: ajoute le workflow de tests"
git branch -M main
git remote add origin https://github.com/VOTRE-COMPTE/atelier-tests.git
git push -u origin main
```

Ouvrir l'onglet **Actions** du dépôt : le job `unit` doit passer au vert. Le job `integration` n'apparaît pas encore : il attend une pull request.

Déclencher le job d'intégration :

```
terminal
git checkout -b feature/verifie-ci
git commit --allow-empty -m "chore: declenche la pull request"
git push -u origin feature/verifie-ci
```

Sur GitHub, cliquer sur « Compare & pull request » puis « Create pull request ». Les deux jobs `unit` et `integration` s'exécutent. Attendre la coche verte et faire une capture d'écran.

Point de contrôle

- ☐ `npm run test:int` affiche **4 tests verts** en local.
- ☐ Le workflow GitHub Actions est **vert** avec les jobs `unit` et `integration`.
- ☐ Vous savez expliquer pourquoi le job d'intégration ne tourne pas sur un simple push.

Questions

1. Pourquoi utiliser `--runInBand` pour les tests d'intégration ?
2. Que se passerait-il sans le `beforeEach` qui recrée les données ?

En cas de problème

Symptôme	Cause probable	Solution
`ECONNREFUSED 127.0.0.1:5432`	Le conteneur n'est pas démarré ou pas encore prêt.	Lancer `docker compose ps`, puis `docker compose up -d` ; attendre quelques secondes.
`port is already allocated`	Un PostgreSQL local utilise déjà le port 5432.	Arrêter le service local, ou changer le mapping en `5433:5432` et `DATABASE_URL`.
`relation "users" does not exist`	Le `beforeEach` manque ou une requête a échoué.	Vérifier que le `CREATE TABLE` est bien dans le `beforeEach`.
`Jest did not exit one second after...`	Les connexions à la base ne sont pas fermées.	Ajouter `afterAll(() => db.end())`.
Job `integration` absent sur GitHub	Aucune pull request ouverte (condition `if`).	Pousser une branche et ouvrir une pull request.
Job d'intégration rouge sur GitHub	Le service PostgreSQL n'est pas prêt ou la variable `DATABASE_URL` diffère.	Vérifier les options `health-*` et que la base se nomme `testdb`.

3 Test fonctionnel de bout en bout

Outil : Playwright

Le test fonctionnel adopte le **point de vue de l'utilisateur** : un vrai navigateur ouvre la page, remplit un formulaire et vérifie ce qui s'affiche. Ces tests sont lents et plus fragiles : on les réserve aux parcours critiques.

À retenir

- Un test E2E traverse **toute la pile** : page web, API, et la base si elle est utilisée.
- On teste le parcours réussi et **un seul** parcours d'échec important, pas toutes les variantes (les niveaux inférieurs s'en chargent).
- On cible les éléments par leur `id` ou leur rôle, pas par leur position à l'écran.

Étape 1 · Ajouter la route de connexion

Dans `src/app.js`, ajouter ce bloc **avant** la ligne `module.exports = app;` :

```
src/app.js (à ajouter avant module.exports)
app.post('/login', (req, res) => {
  const { email, password } = req.body;
  if (email === 'a@test.fr' && password === 'secret') {
    return res.sendStatus(200);
  }
  res.sendStatus(401);
});
```

Attention

- Les identifiants sont codés en dur **uniquement pour cet atelier**. C'est exactement ce que le Bloc C (gestion des secrets) apprend à corriger.

Vérifier la route à la main avec `npm start` puis, dans un second terminal :

```
terminal
curl -i -X POST http://localhost:3000/login \
  -H "Content-Type: application/json" \
  -d '{"email":"a@test.fr","password":"secret"}'
```

La première ligne de la réponse doit être `HTTP/1.1 200 OK`. Avec un mauvais mot de passe, elle doit être `HTTP/1.1 401 Unauthorized`. (Sous Windows PowerShell, utilisez plutôt le navigateur à l'étape suivante.)

Étape 2 · Créer les pages web

Créer `public/login.html` : un formulaire qui envoie les identifiants à `/login`, redirige vers le tableau de bord en cas de succès, et affiche un message sinon.

```
public/login.html
<form id="f">
  <input id="email" type="email" placeholder="Email">
  <input id="password" type="password" placeholder="Mot de passe">
  <button type="submit">Se connecter</button>
  <p id="erreur"></p>
</form>

<script>
document.getElementById('f').addEventListener('submit', async (e) => {
  e.preventDefault();
  const r = await fetch('/login', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ email: email.value, password: password.value })
  });
  if (r.ok) location.href = '/dashboard.html';
  else document.getElementById('erreur').textContent = 'Identifiants invalides';
});
</script>
```

```
public/dashboard.html
<h1>Dashboard</h1>
```

Essai manuel : lancer `npm start`, ouvrir `http://localhost:3000/login.html` dans le navigateur, se connecter avec `a@test.fr` / `secret`, puis avec un mauvais mot de passe. Vous devez retrouver les deux comportements que Playwright va automatiser. Arrêter le serveur ensuite.

Étape 3 · Configurer Playwright

Créer `playwright.config.js`. L'option `webServer` démarre l'application automatiquement avant les tests ; `baseURL` évite de répéter l'adresse ; le reporter HTML permet de consulter un rapport.

```
playwright.config.js
module.exports = {
  testDir: './e2e',
  reporter: [['list'], ['html', { open: 'never' }]],
  use: { baseURL: 'http://localhost:3000' },
  webServer: {
    command: 'node src/server.js',
    url: 'http://localhost:3000',
    reuseExistingServer: true,
  },
};
```

Télécharger le navigateur de test (une seule fois, environ 150 Mo) :

```
terminal
npx playwright install chromium
```

Étape 4 · Écrire le scénario de connexion réussie

Créer `e2e/login.spec.js`. Le scénario reproduit exactement ce que vous avez fait à la main.

```
e2e/login.spec.js
const { test, expect } = require('@playwright/test');

test('connexion réussie', async ({ page }) => {
  await page.goto('/login.html');
  await page.fill('#email', 'a@test.fr');
  await page.fill('#password', 'secret');
  await page.click('button[type=submit]');

  await expect(page).toHaveURL('/dashboard.html');
});
```

```
terminal
npm run test:e2e
```

```
résultat attendu (indicatif)
Running 1 test using 1 worker
  ✓  1 e2e/login.spec.js:3:1 › connexion réussie (650ms)

  1 passed (2s)
```

Étape 5 · Écrire le scénario de connexion échouée

Ajouter à la suite du fichier un second test : avec un mauvais mot de passe, le message d'erreur s'affiche et l'URL ne change pas.

```
e2e/login.spec.js (à ajouter à la suite)
test('connexion échouée', async ({ page }) => {
  await page.goto('/login.html');
  await page.fill('#email', 'a@test.fr');
  await page.fill('#password', 'mauvais');
  await page.click('button[type=submit]');

  await expect(page.locator('#erreur')).toHaveText('Identifiants invalides');
  await expect(page).toHaveURL('/login.html');
});
```

```
terminal
npm run test:e2e
```

Vous devez voir **2 tests verts**.

Étape 6 · Consulter le rapport et observer le navigateur

```
terminal
npx playwright show-report
npx playwright test --headed
```

La première commande ouvre le rapport HTML dans le navigateur (à capturer pour le livrable). La seconde relance les tests en **affichant la fenêtre du navigateur** : vous voyez la page se remplir toute seule.

Commit conseillé : `git add . && git commit -m "test: ajoute les scenarios e2e de connexion"`

Étape 7 · Comparer les trois niveaux de tests

Relancer successivement `npm run test:unit`, `npm run test:int` et `npm run test:e2e`. Relever la durée affichée par chaque outil (ligne « Time » pour Jest, « passed (…s) » pour Playwright), puis compléter ce tableau dans votre compte rendu.

	Unitaires	Intégration	E2E
Durée mesurée			
Ce qui est vérifié			
Dépendances nécessaires			
Quand le lancer en CI			

Point de contrôle

- ☐ Les **2 scénarios E2E** (succès et échec) passent.
- ☐ Le rapport Playwright s'ouvre avec `npx playwright show-report`.
- ☐ Le tableau comparatif est rempli avec **vos** durées mesurées.

Question

1. Pourquoi ne teste-t-on pas **tous** les cas (prix négatif, id invalide…) en E2E ?

En cas de problème

Symptôme	Cause probable	Solution
`browserType.launch: Executable doesn't exist`	Le navigateur de test n'est pas installé.	Lancer `npx playwright install chromium`.
`Timed out waiting from config.webServer`	Le serveur ne démarre pas (erreur de code ou port 3000 occupé).	Lancer `node src/server.js` seul pour lire l'erreur ; arrêter tout autre serveur sur le port 3000.
`page.fill: Timeout ... waiting for locator('#email')`	La page n'est pas celle attendue ou l'`id` est faux.	Ouvrir `/login.html` à la main ; vérifier `public/` et le serveur lancé à la racine du projet.
Jest lance aussi les tests Playwright	La section `jest` du `package.json` est absente ou mal écrite.	Vérifier `testPathIgnorePatterns`.
`Cannot GET /login.html`	`express.static('public')` ne trouve pas le dossier.	Lancer `npm start` depuis la racine du projet.

Livrable et évaluation

À déposer à la fin de l'atelier

Livrable

- ☐ **Dépôt Git** avec un historique de commits lisible (un commit par étape). Fournir le lien du dépôt GitHub.
- ☐ **Capture de la couverture ≥ 80 %** : sortie console de `npm run test:unit`.
- ☐ **Capture du workflow GitHub Actions vert**, avec les jobs `unit` et `integration` visibles.
- ☐ **Rapport Playwright** : capture de `npx playwright show-report`.
- ☐ **Tableau comparatif** des 3 niveaux, rempli avec vos durées mesurées.
- ☐ **Réponses aux 5 questions**, en 2 ou 3 lignes chacune.

Barème indicatif

Critère	Points
Tests unitaires + mock corrects, couverture ≥ 80 %	5
Tests d'intégration (200, 404, 400, injection) avec données recréées	5
Scénarios E2E (succès et échec)	3
Workflow CI : jobs séparés, service PostgreSQL	4
Tableau comparatif et réponses aux questions	3
Total	**20**

Pour aller plus loin

- Ajouter un job E2E déclenché uniquement sur la branche **staging** (installer Chromium avec `npx playwright install --with-deps chromium`).
- Faire échouer le pipeline si la couverture passe sous 80 % (option `coverageThreshold` de Jest).
- Suite : Atelier 2 (Bloc B) avec SonarQube, l'analyse des dépendances et Trivy sur ce même projet.